

Kontextfreie Sprachen III – Kellerautomaten

(Contextfree Languages and Pushdown Automata)

Andreas Schäfer

Theoretische Informatik

FH Lübeck

Kaffeeautomaten

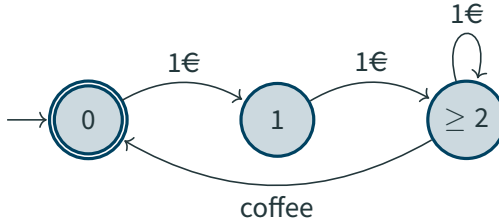


Abbildung 1: Kaffeeautomat 1.0

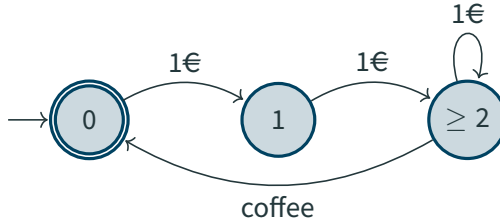


Abbildung 1: Kaffeeautomat 1.0

- Nutzer muss mindestens 2 € einwerfen
- Automat gibt kein Wechselgeld

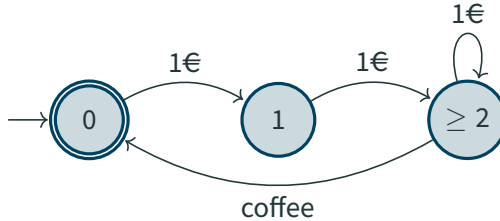


Abbildung 1: Kaffeeautomat 1.0

- Nutzer muss mindestens 2 € einwerfen
- Automat gibt kein Wechselgeld
- Was ist das Problem mit dem Wechselgeld?



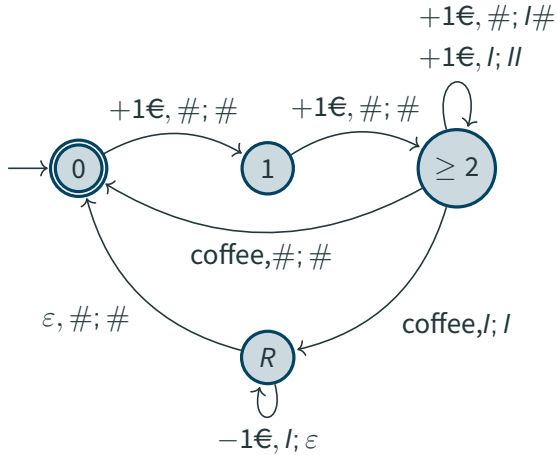
- Endlicher Automat merkt sich gezahltes Geld im Zustand
- Endlicher Automat hat nur endlich viele Zustände
- Endlicher Automat kann also nur bis zu einer Obergrenze zählen
- Wenn Nutzer mehr Geld einwirft, kommt Automat nicht mehr mit

- Endlicher Automat merkt sich gezahltes Geld im Zustand
- Endlicher Automat hat nur endlich viele Zustände
- Endlicher Automat kann also nur bis zu einer Obergrenze zählen
- Wenn Nutzer mehr Geld einwirft, kommt Automat nicht mehr mit
- Lösung?

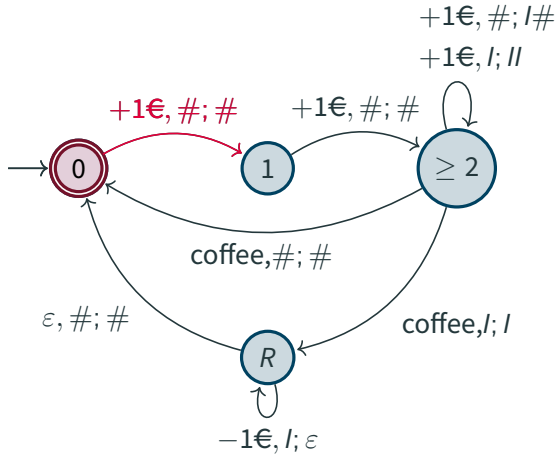


- Endlicher Automat merkt sich gezahltes Geld im Zustand
- Endlicher Automat hat nur endlich viele Zustände
- Endlicher Automat kann also nur bis zu einer Obergrenze zählen
- Wenn Nutzer mehr Geld einwirft, kommt Automat nicht mehr mit
- Lösung: Zusätzlicher Speicherplatz (Bierdeckelnotation)

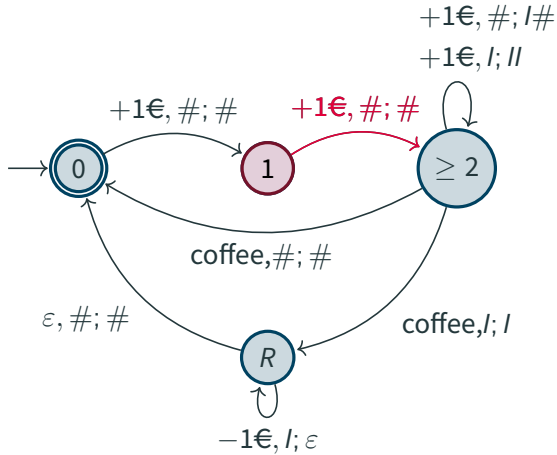
KAFFEEAUTOMAT 2.0



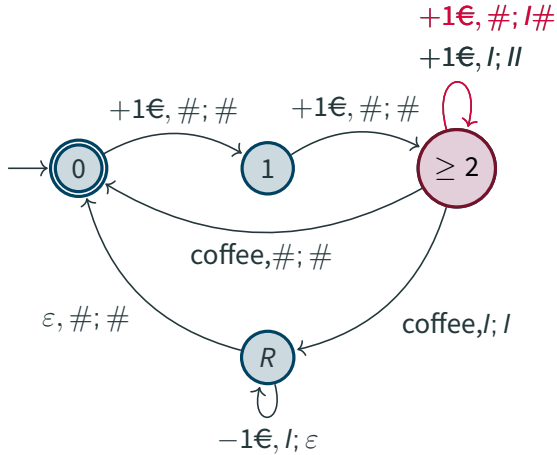
KAFFEEAUTOMAT 2.0



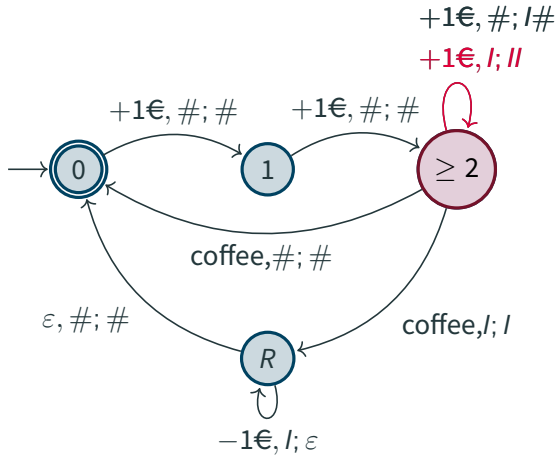
KAFFEEAUTOMAT 2.0



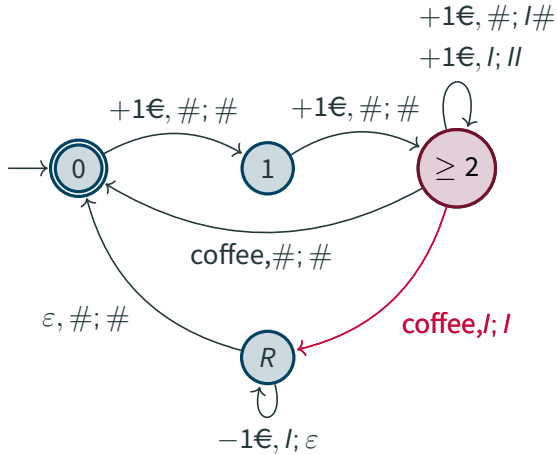
KAFFEEAUTOMAT 2.0



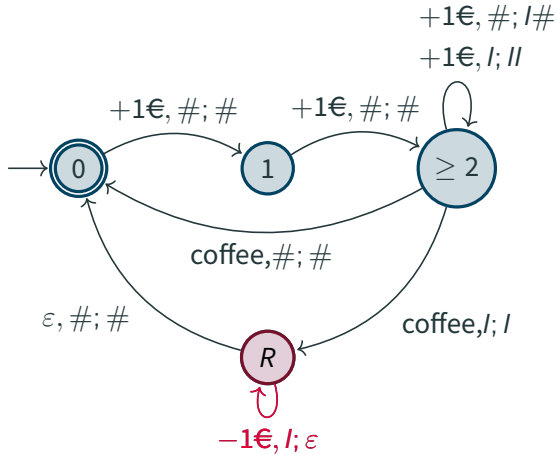
KAFFEEAUTOMAT 2.0



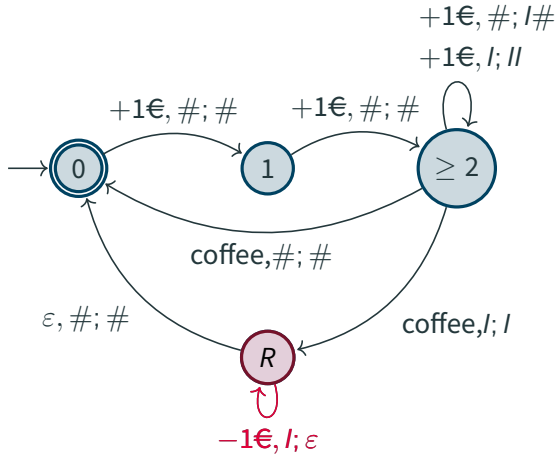
KAFFEEAUTOMAT 2.0



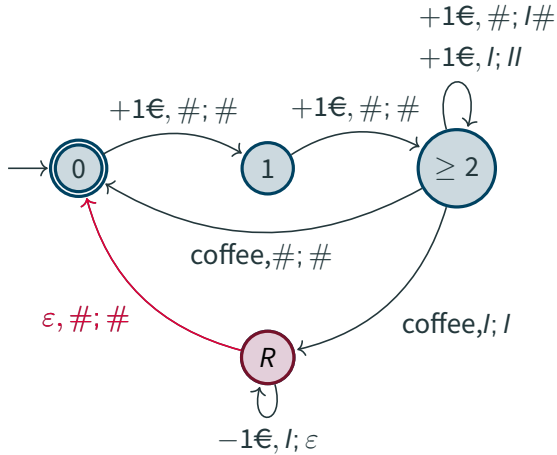
KAFFEEAUTOMAT 2.0



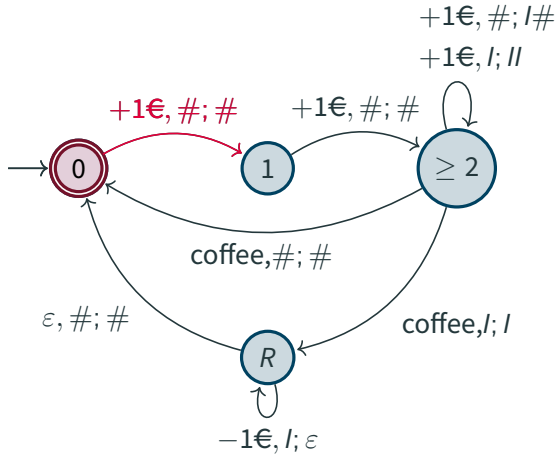
KAFFEEAUTOMAT 2.0



KAFFEEAUTOMAT 2.0



KAFFEEAUTOMAT 2.0

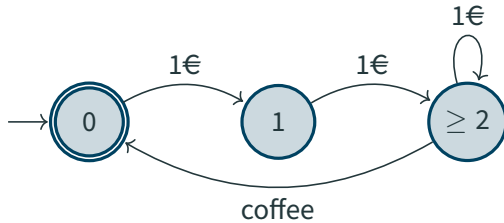




- Zentrale Frage: Was können Computer berechnen?

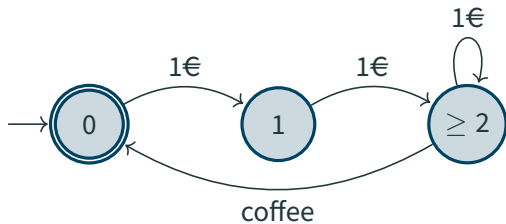


- Formales Modell eines Computers: Turing-Maschine
 - Vorteil: Kann alles berechnen was ein Computer berechnen kann
 - Aber: Es gibt viele Probleme, die auch Turing-Maschinen nicht lösen können
 - Nachteil: Turing-Maschinen und deren Verhalten sind – wie bei normalen Programmen – schwer zu analysieren
- Idee: Einfachere Automatenmodelle
 - Reichen für einfache Probleme aus
 - Sind besser oder sogar gut zu analysieren
 - Sind auch einfacher zu realisieren in HW/SW

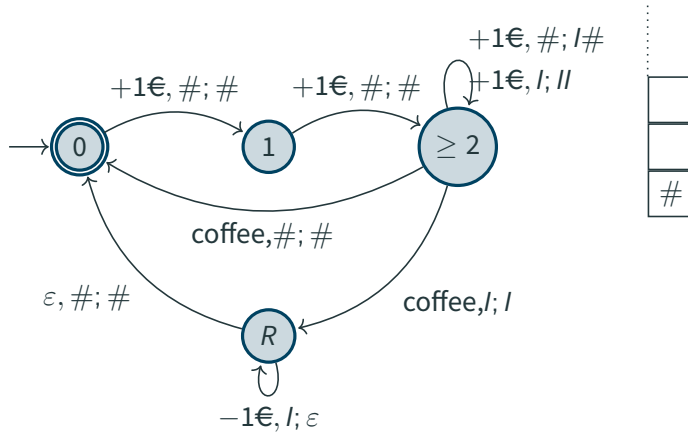


Kann man einen Kaffee bekommen?

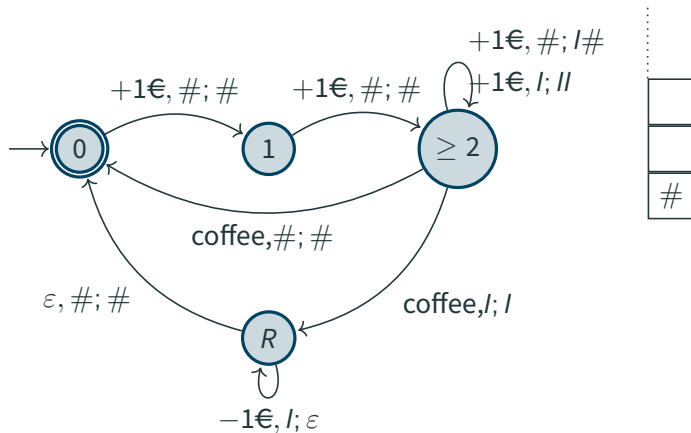




Kann man einen Kaffee bekommen? Ja, leicht zu sehen



Kann man einen Kaffee bekommen?



Kann man einen Kaffee bekommen?

Kellerautomaten

WAS IST EIN KELLER (STACK)?

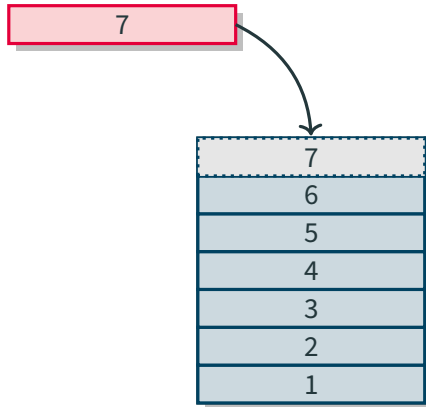


WAS IST EIN KELLER (STACK)?

- Datenstruktur bei der nur auf das oberste / zuletzt eingefügte Element zugegriffen werden kann

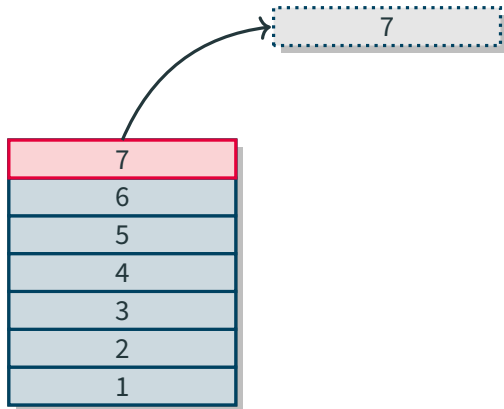
WAS IST EIN KELLER (STACK)?

- Datenstruktur bei der nur auf das oberste / zuletzt eingefügte Element zugegriffen werden kann
- push legt ein neues Element oben auf den Stack



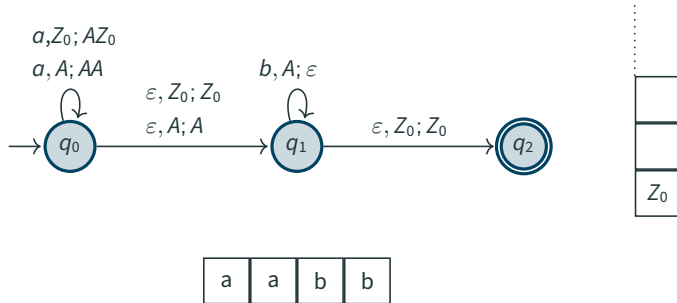
WAS IST EIN KELLER (STACK)?

- Datenstruktur bei der nur auf das oberste / zuletzt eingefügte Element zugegriffen werden kann
- push legt ein neues Element oben auf den Stack
- **pop holt das oberste Element**

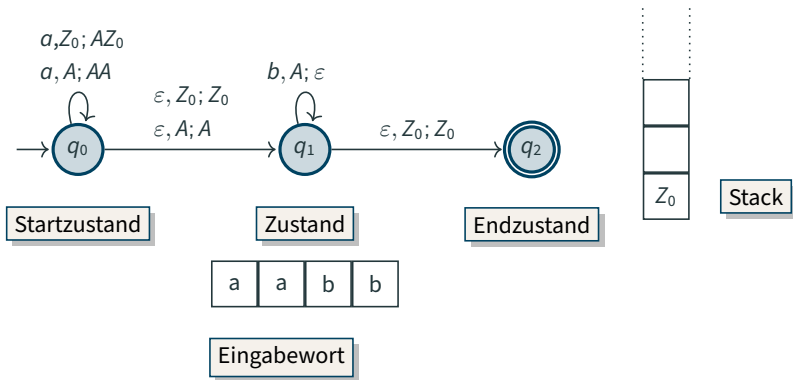


- Kellerautomaten - Automaten zu kontextfreien Sprachen
- Problem bei endlichen Automaten: Hat keinen externen Speicher, muss sich alles im Zustand merken.
- Kellerautomat = Endlicher Automat + Stack
- Liest in jedem Schritt ein Zeichen vom Wort und das oberste Symbol auf dem Stack
- Schreibt in jedem Schritt ein Wort auf den Stack – kann auch das leere Wort sein
- Akzeptiert mit Endzustand

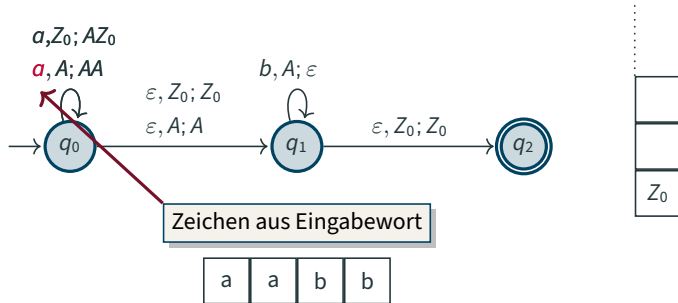
$$a^n b^n$$



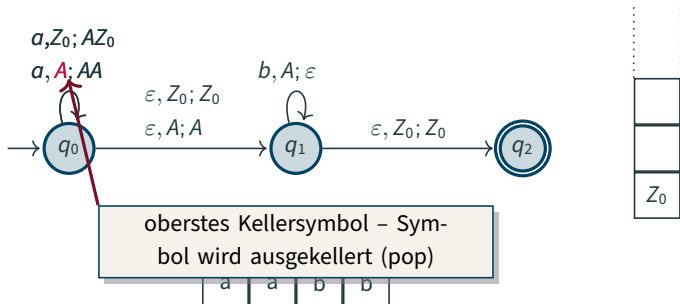
$$a^n b^n$$



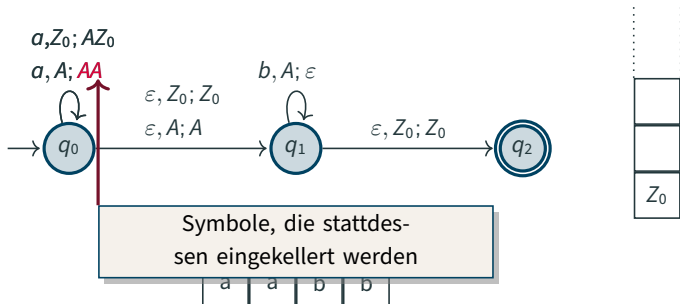
$$a^n b^n$$



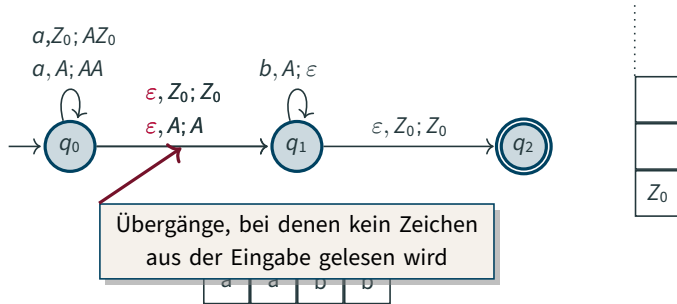
$$a^n b^n$$



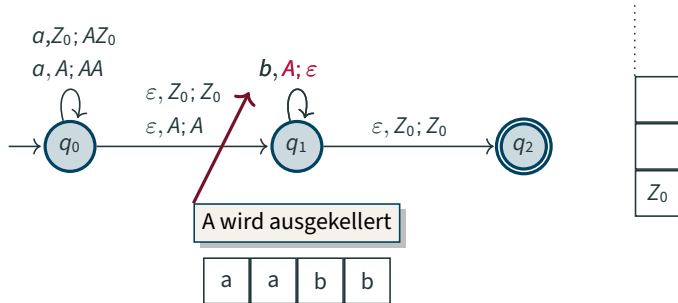
$$a^n b^n$$



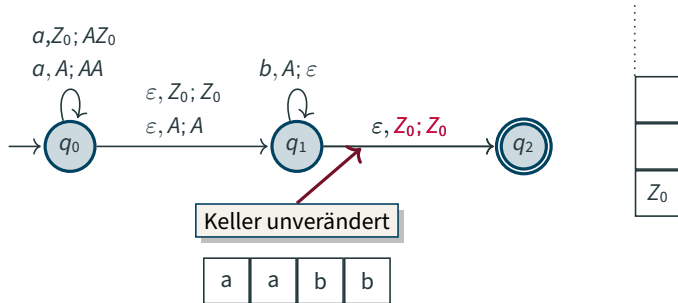
$$a^n b^n$$



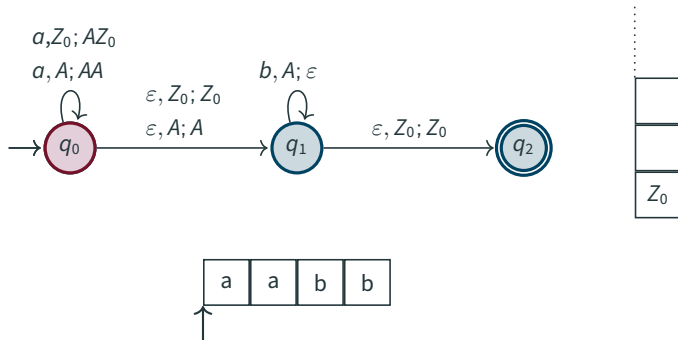
$$a^n b^n$$



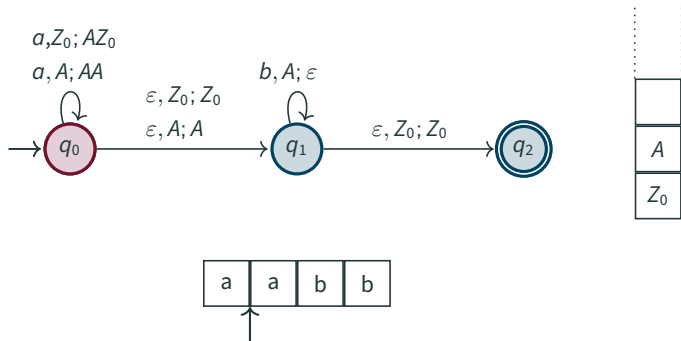
$$a^n b^n$$



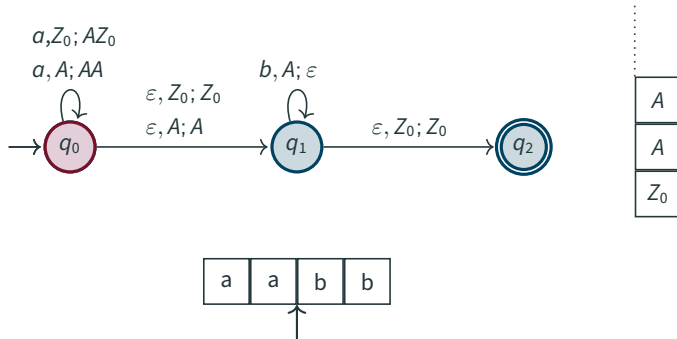
$$a^n b^n$$



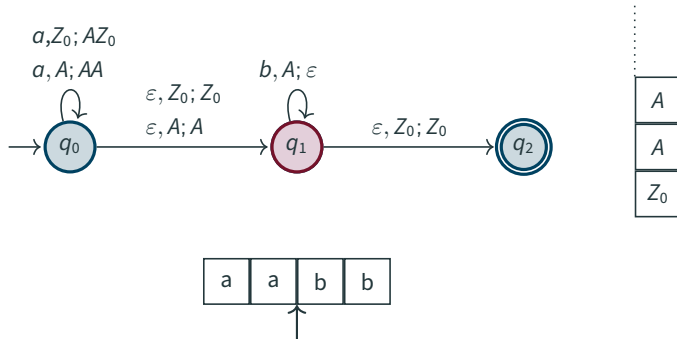
$$a^n b^n$$



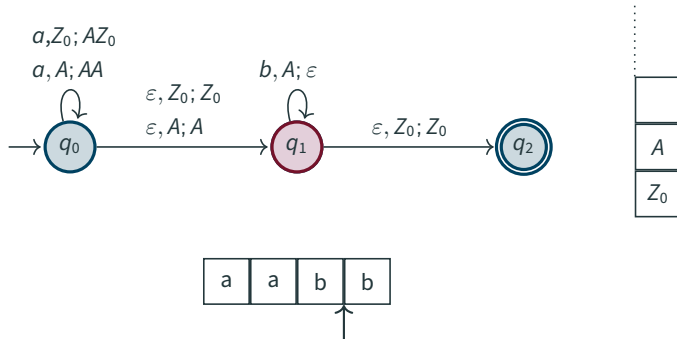
$$a^n b^n$$



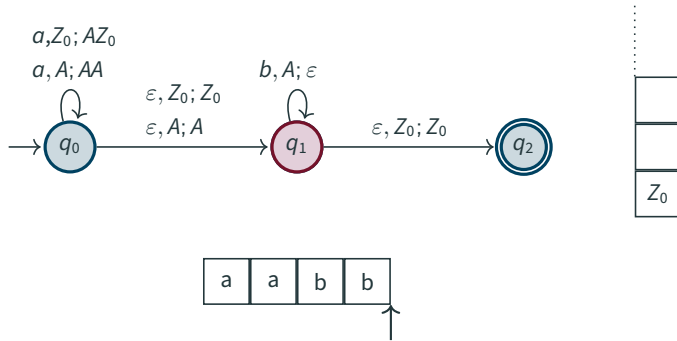
$$a^n b^n$$



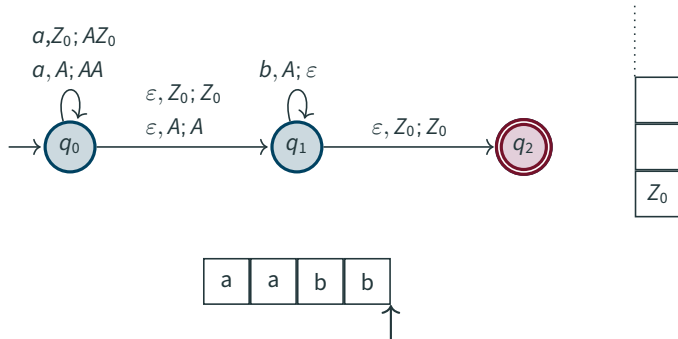
$$a^n b^n$$



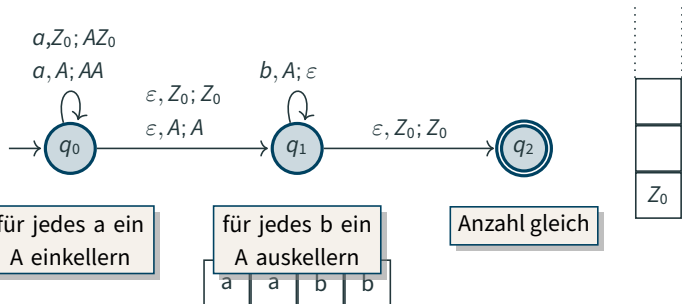
$$a^n b^n$$



$$a^n b^n$$



$$a^n b^n$$



Definition (Kellerautomat (pushdown automaton (PDA)))

Ein nichtdeterministischer Kellerautomat ist ein Tupel $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.

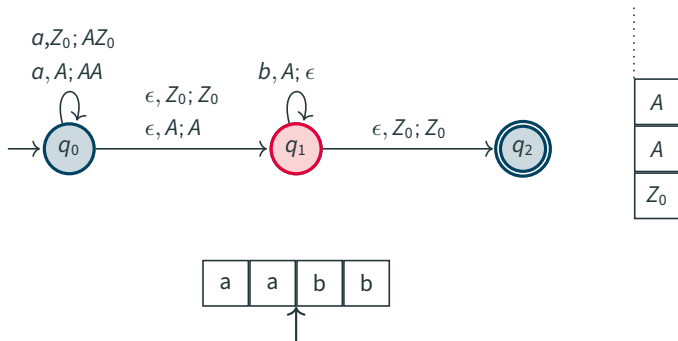
- Q ist eine endliche Menge von **Zuständen** (states)
- Σ ist eine endliche Menge, das **Eingabealphabet** (input alphabet)
- Γ ist eine endliche Menge, das **Kelleralphabet** (stack alphabet)
- $\delta : (Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma) \rightarrow \mathcal{P}(Q \times \Gamma^*)$ ist die **Überföhrungsfunktion** (transition function)
- $q_0 \in Q$ ist der **Anfangszustand** (start / initial state)
- Z_0 ist das oberste Kellersymbol beim Start
- $F \subseteq Q$ ist die Menge der **Endzustände** (final / accepting states)

$$\delta : (Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma) \rightarrow \mathcal{P}(Q \times \Gamma^*)$$

$$\delta(q, a, A) \ni (q', B_1 \dots B_n) \text{ oder } \delta(q, \epsilon, A) \ni (q', B_1 \dots B_n)$$

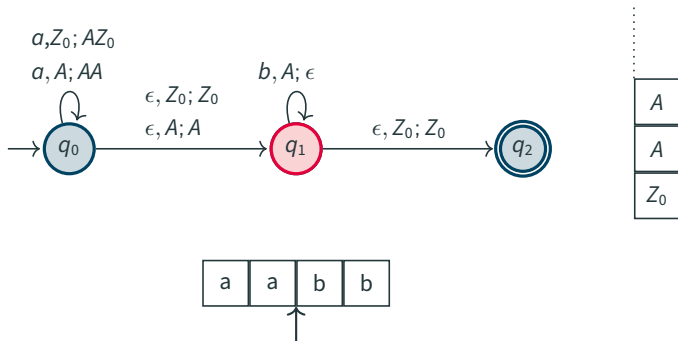
- Ausgangszustand q
- Ein Symbol des Eingabewortes a lesen oder ϵ , d.h. Transition ohne etwas vom Wort zu lesen
- Oberstes Kellersymbol A vom Stack poppen – **notwendig**
- Nachfolgezustand q'
- Wort $B_1 \dots B_n$ auf den Stack schreiben – kann auch das leere Wort sein

KONFIGURATION (INSTANTANEOUS DESCRIPTION)



- Graphische Darstellung aufwändig

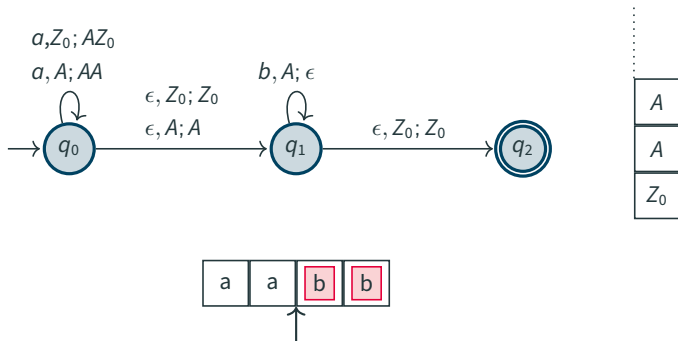
KONFIGURATION (INSTANTANEOUS DESCRIPTION)



- Es reicht sich zu merken: Zustand

$(q_1,$

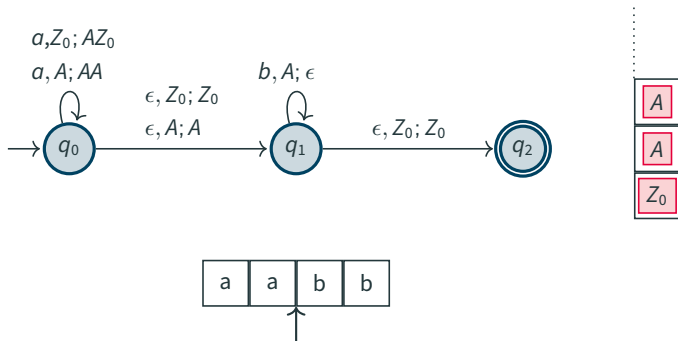
KONFIGURATION (INSTANTANEOUS DESCRIPTION)



- Es reicht sich zu merken: Zustand, Rest des Eingabeworts

$(q_1, bb$

KONFIGURATION (INSTANTANEOUS DESCRIPTION)



- Es reicht sich zu merken: Zustand, Rest des Eingabeworts, Kellerinhalt

(q_1, bb, AAZ_0)

Definition (Konfiguration (instantaneous description))

Eine Konfiguration $(q, a_1 \dots a_n, A_1 \dots A_k)$ ist ein Tripel aus $Q \times \Sigma^* \times \Gamma^*$ aktuellem Zustand, restlichem Eingabewort und Kellerinhalt.

Definition (Rechenschritt)

Für einen Kellerautomaten P ist $\vdash$ definiert durch

$$(q, a_1 \dots a_n, A_1 \dots A_k) \vdash (q', a_2 \dots a_n, B_1 \dots B_m A_2 \dots A_k) \\ \text{gdw. } \delta(q, a_1, A_1) \ni (q', B_1 \dots B_m)$$

und

$$(q, a_1 \dots a_n, A_1 \dots A_k) \vdash (q', a_1 \dots a_n, B_1 \dots B_m A_2 \dots A_k) \\ \text{gdw. } \delta(q, \epsilon, A_1) \ni (q', B_1 \dots B_m)$$

Notation

Wir schreiben $(q, w, \alpha) \vdash^* (q', w', \alpha')$ falls die Konfiguration (q', w', α') in endlich vielen Schritten aus (q, w, α) erreichbar ist.

Definition (Akzeptanz mit Endzustand)

Sei $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ ein Kellerautomat. P akzeptiert ein Wort $w \in \Sigma^*$ mit Endzustand falls $(q_0, w, Z_0) \vdash^* (q_f, \epsilon, \alpha)$ und $q_f \in F$, also falls es die Möglichkeit gibt, dass der Kellerautomat das ganze Eingabewort vollständig abarbeitet und sich dann in einem Endzustand befindet.

Die mit Endzustand akzeptierte Sprache ist

$$L(P) = \{w \in \Sigma^* \mid P \text{ akzeptiert } w \text{ mit Endzustand}\}.$$

Definition (Akzeptanz mit leerem Keller)

Sei $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ ein Kellerautomat. P akzeptiert ein Wort $w \in \Sigma^*$ mit leerem Keller falls $(q_0, w, Z_0) \vdash^* (q', \epsilon, \epsilon)$, also falls der Kellerautomat bei Abarbeiten des Wortes den Keller leeren kann. Beachte, dass ein Kellerautomat mit leerem Keller keine Schritte machen kann.

Die mit leerem Keller akzeptierte Sprache ist dann

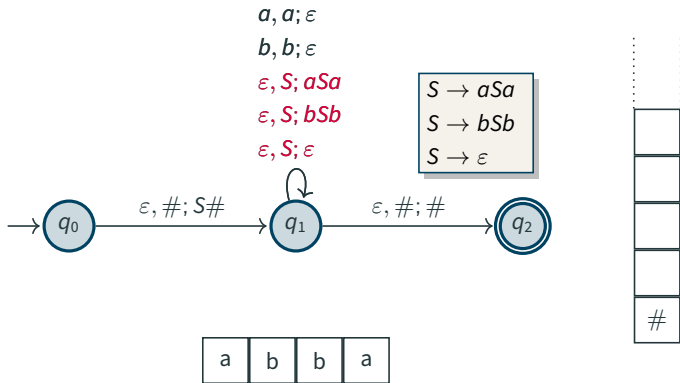
$$N(P) = \{w \in \Sigma^* \mid P \text{ akzeptiert } w \text{ mit leerem Keller}\}.$$

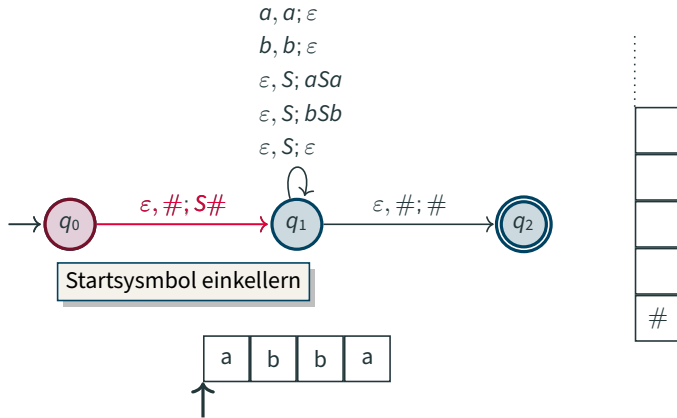
Aufgabe

- Konstruieren Sie einen Kellerautomaten, der die Sprache $\{wcw^R \mid w \in \{a, b\}^*\}$ akzeptiert.
- Konstruieren Sie einen Kellerautomaten, der die Sprache $\{ww^R \mid w \in \{a, b\}^*\}$ akzeptiert.
(Nutzen Sie nichtdeterministisches Raten)

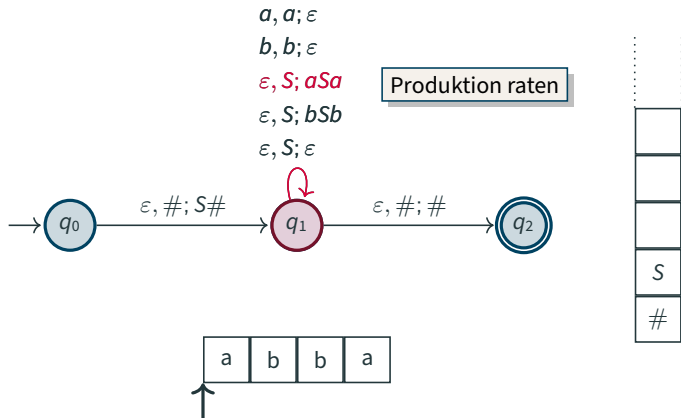
- Kellerautomaten: *nichtdeterministisch* mit ε -Übergängen
- Spezialfall deterministischer Kellerautomat: für jede Kombination aus Zustand und Zeichen gibt es maximal einen Nachfolger oder einen ε -Nachfolger
- Deterministische Kellerautomaten sind echte Teilklasse der nichtdeterministischen Kellerautomaten
- Für ww^R gibt es keinen deterministischen Kellerautomaten, die Mitte muss geraten werden.

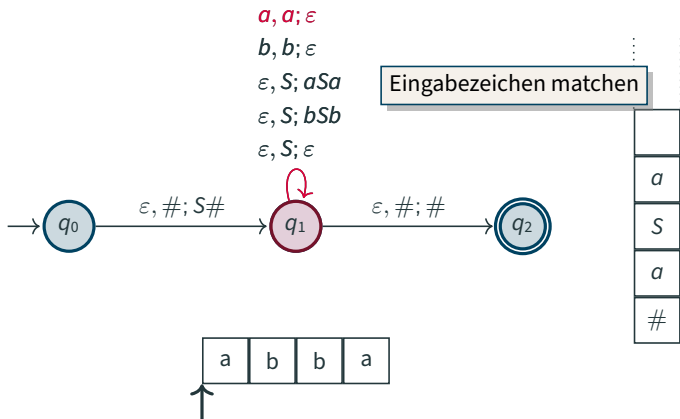
- Zu kontextfreier Grammatik $G = (N, \Sigma, P, S)$ Kellerautomat P konstruieren.
- Idee: Ableitung im Keller simulieren
- Wenn Terminalzeichen Kellertop, dann matching
- Wenn Nichtterminal Kellertop, dann Produktion anwenden (nichtdeterministisch raten)



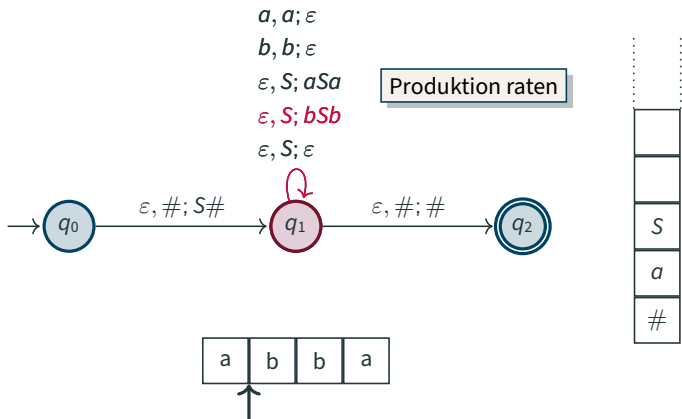


GRAMMATIK $\rightarrow$ PDA

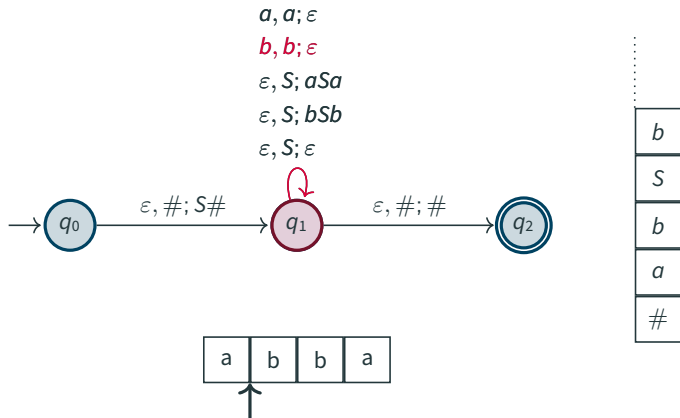




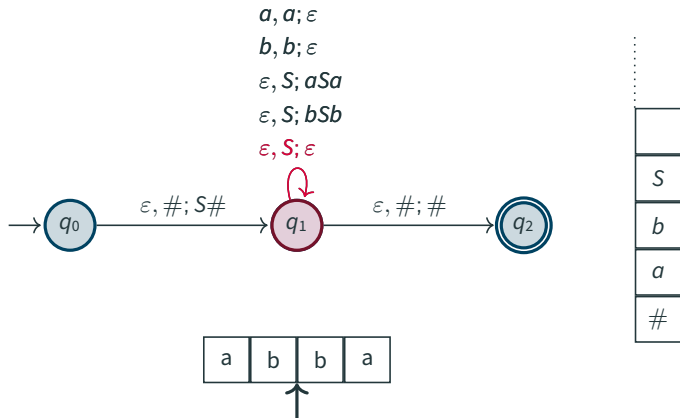
GRAMMATIK $\rightarrow$ PDA



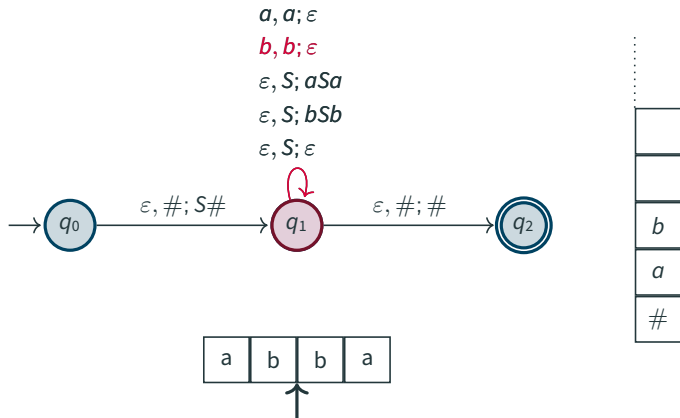
GRAMMATIK $\rightarrow$ PDA



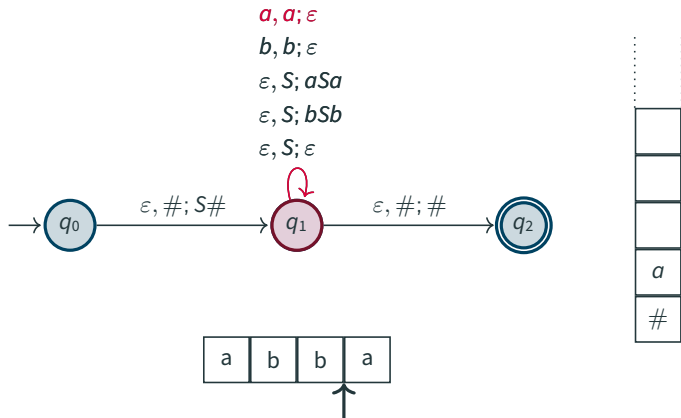
GRAMMATIK $\rightarrow$ PDA



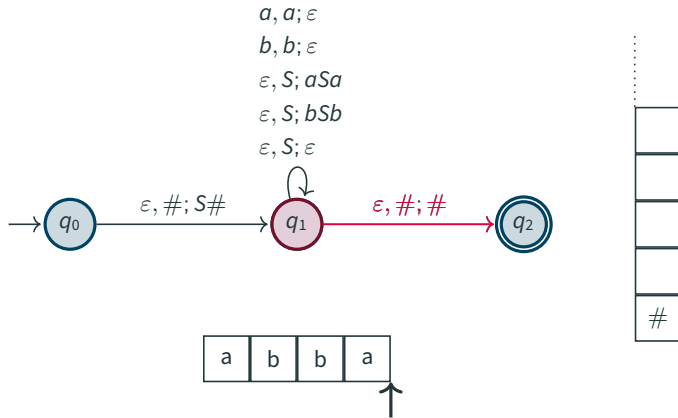
GRAMMATIK $\rightarrow$ PDA



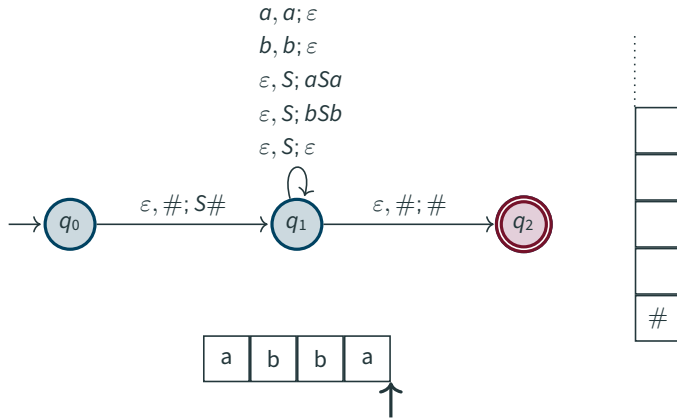
GRAMMATIK $\rightarrow$ PDA



GRAMMATIK $\rightarrow$ PDA



GRAMMATIK $\rightarrow$ PDA



- Gegeben kontextfreie Grammatik $G = (N, \Sigma, R, S)$
- Konstruiere PDA $P = (\{q_0, q_1, q_F\}, \Sigma, N \cup \Sigma \cup \{\#\}, \delta, q_0, \#, \{q_F\})$
 - 3 Zustände
 - Eingabealphabet wie bei G
 - Kelleralphabet alle Terminale, Nichtterminale und ein frisches Zeichen $\#$
 - $\#$ ist initial im Keller

- Gegeben kontextfreie Grammatik $G = (N, \Sigma, R, S)$
- Konstruiere PDA $P = (\{q_0, q_1, q_F\}, \Sigma, N \cup \Sigma \cup \{\#\}, \delta, q_0, \#, \{q_F\})$
 - An Anfang Startsymbol auf Keller legen
$$\delta(q_0, \varepsilon, \#) = \{(q_1, S\#)\}$$
 - An Ende in Endzustand wechseln wenn nur noch $\#$ im Keller
$$\delta(q_1, \varepsilon, \#) = \{(q_F, \varepsilon)\}$$
 - Wenn Terminal Kellertop, Eingabe matchen
$$\delta(q_1, a, a) = \{(q_1, \varepsilon)\} \text{ für alle } a \in \Sigma$$
 - Wenn Nichtterminal Kellertop, Produktion anwenden
$$\delta(q_1, \varepsilon, A) = \{(q_1, B_1 \dots B_n) \mid A \rightarrow B_1 \dots B_n \in R\} \text{ für alle Produktionen der Grammatik } A \rightarrow B_1 \dots B_n$$

- Für jeden PDA kann auch eine kontextfreie Grammatik konstruiert werden, die die gleiche Sprache erzeugt.
- Konstruktion: Technisch
- siehe: Schöning, Seite 73, Hopcroft, Motwani, Ullmann, Seite 242

- Kontextfreie Grammatiken und Sprachen
- Abschluss unter Vereinigung, Konkatenation und Stern
- Pumping-Lemma
- Nicht-Abschluss unter Schnitt und Komplement
- Kellerautomaten als *das* Automatenmodell zu kontextfreien Sprachen